CNN, RNN and LSTM: Interview Mind Map

Oct 20, 2026 · @Mukundan

Overview

CNN, RNN and LSTM are three foundational neural network families. CNNs handle data with spatial structure, mainly images. RNNs handle sequences where order matters. LSTM is a specific, improved type of RNN built to remember information over long sequences.

The one line to anchor the whole map: CNN sees space, RNN remembers time, LSTM remembers time well.

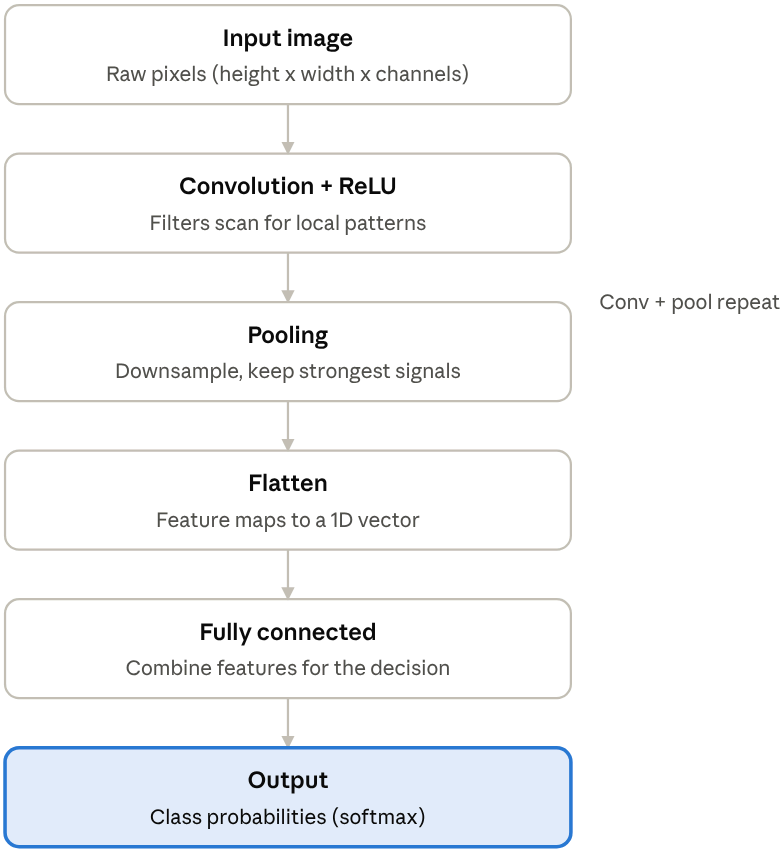
CNN (Convolutional Neural Network)

Root idea: finds patterns that can appear anywhere in an image.

1. Convolution: a small filter (kernel) slides across the image computing dot products. Each filter detects one pattern like an edge, curve or texture. The output is a feature map.
2. Stride and padding: stride is how far the filter jumps each step. Padding adds a border so edge pixels are not lost.
3. Activation (ReLU): adds non-linearity, keeps positive signals, zeroes the rest.
4. Pooling: max pooling takes the strongest value in a patch. Shrinks size and adds position tolerance.
5. Hierarchy: early layers catch edges, middle layers catch shapes, deep layers catch whole objects.
6. Flatten plus dense: final feature maps flatten into a normal network that outputs the prediction.
7. Why it wins: parameter sharing (same filter reused everywhere) means far fewer weights, and translation invariance (a cat is a cat wherever it sits).

Say it: "A CNN slides filters over an image to detect local patterns, pools to keep the strongest ones, and stacks layers so it learns edges, then shapes, then objects."

CNN: an image flows through conv and pool layers to a class



CNN architecture · input image to class output

Reading the figure, top to bottom:

- Input image: the raw pixel grid with its channels (3 for RGB). Just data, nothing learned yet.
- Convolution + ReLU: filters slide over the input to make feature maps, then ReLU zeroes negatives to add non-linearity.
- Pooling: max pooling keeps the strongest value in each small patch, shrinking the map and adding position tolerance.
- Flatten: the final feature maps are unrolled into one long vector a standard network can read.
- Fully connected: dense layers mix all features to weigh the evidence for each class.
- Output: softmax turns the scores into class probabilities that sum to 1.

The convolution and pooling block usually repeats several times, building features from edges to shapes to whole objects.

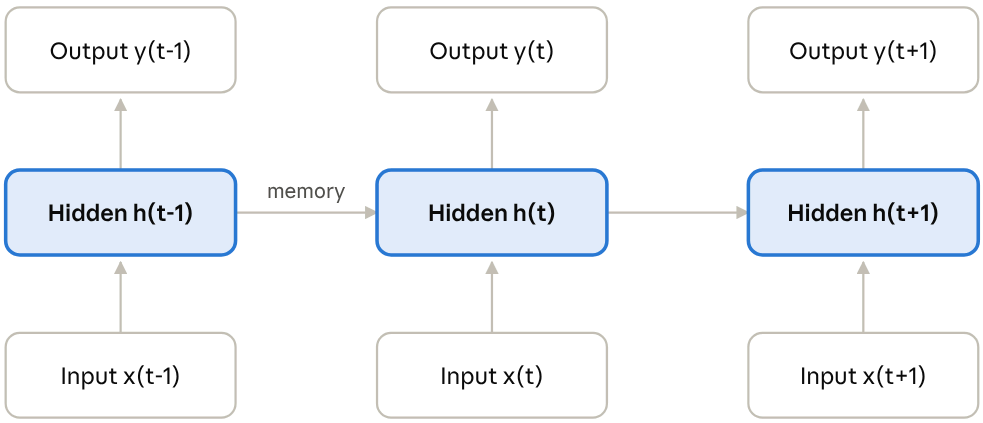
RNN (Recurrent Neural Network)

Root idea: processes a sequence one step at a time and carries memory forward.

1. Hidden state: a vector updated at each step. It is the running memory of everything seen so far.
2. Weight sharing across time: the same weights are applied at every step, which is why it handles variable-length input.
3. Unrolling: picture the RNN copied once per time step, chained together.
4. BPTT: training uses backpropagation through time, flowing gradients backward across all steps.
5. Core weakness: vanishing gradients. Over long sequences gradients shrink toward zero, so early information is forgotten. Exploding gradients is the opposite case, less common.
6. Sequence shapes: many-to-one (sentiment), one-to-many (captioning), many-to-many (translation).

Say it: "An RNN reads a sequence step by step, keeping a hidden state as memory. It works for short sequences but forgets long-range context because gradients vanish."

RNN unrolled: the hidden state carries memory across steps



RNN unrolled across three time steps

The figure shows one RNN cell unrolled across time. The same cell is reused at every step.

- Input x(t): the element at the current step, like one word or one time point.
- Hidden state h(t): computed from the current input and the previous hidden state h(t-1). This is the memory, and it is why the three middle boxes are highlighted.

- Recurrence (horizontal arrows): the hidden state is passed forward, so step t carries context from everything before it.
- Output y(t): produced from the current hidden state. Some tasks use every output, some only the last one.
- Shared weights: the same weight matrices are applied at every step, which is why the structure repeats.

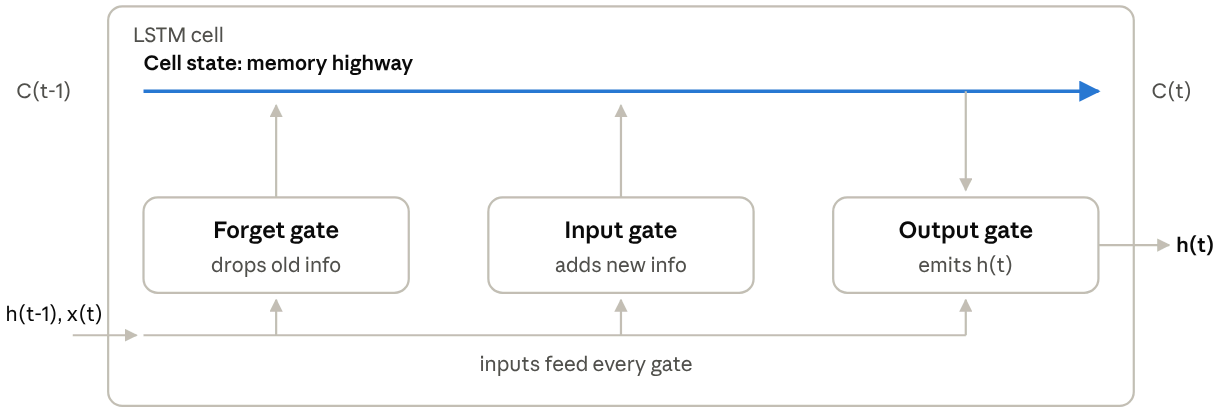
LSTM (Long Short-Term Memory)

Root idea: an RNN redesigned so memory survives over long sequences.

1. Cell state: the memory highway. Information flows along it mostly unchanged, which is what fixes the vanishing gradient problem.
2. Three gates, each a sigmoid with output between 0 and 1:
   - Forget gate: decides what to erase from the cell state.
   - Input gate: decides what new information to write.
   - Output gate: decides what to expose as this step's hidden state.
3. Candidate values: a tanh layer proposes new content, and the input gate decides how much gets in.
4. GRU (good to name): a lighter cousin with two gates and no separate cell state. Faster, often similar accuracy.

Say it: "An LSTM adds a cell state that carries memory across many steps, controlled by forget, input and output gates. The gates let it hold onto what matters and drop what does not, which solves the RNN forgetting problem."

Inside an LSTM: three gates regulate the cell state



LSTM cell · cell state and the three gates

This is the inside of a single LSTM cell at one time step.

- Cell state: the highlighted highway across the top. Information rides along it with only minor linear edits, which is how gradients survive over long sequences.
- Forget gate: reads h(t-1) and x(t) and outputs a 0 to 1 value per element, deciding how much of the old cell state to keep.
- Input gate: decides which new candidate values (from a tanh layer) get written into the cell state.
- Output gate: decides which parts of the updated cell state become the hidden state h(t), the cell's output for this step.
- Inputs: every gate reads the previous hidden state h(t-1) and the current input x(t), shown by the feed line at the bottom.

The comparison

This is the payoff that interviewers want.

Architecture	Best for	Core trick
CNN	Images, grids	Filters detect local patterns
RNN	Sequences	Hidden state carries memory forward
LSTM	Long sequences	Gates protect long-term memory

Two more contrasts worth saying:

- CNN processes in parallel. RNN and LSTM process sequentially.
- All three are foundations. Transformers now dominate sequence and language tasks because they capture long-range context without stepping through time, and they train faster. CNNs still rule most vision, though Vision Transformers are catching up.

Common interview follow-ups

- Why does a CNN use fewer parameters than a dense network? Parameter sharing plus local connections.
- What exactly causes vanishing gradients in an RNN? Repeated multiplication of small gradients through many time steps.
- How does the LSTM cell state avoid that? Additive updates along the cell state, not repeated multiplication.
- When would you still pick an LSTM over a Transformer? Small data, streaming input, or limited compute.